[Dung Vu] Socket Programming - TCP/UDP Sockets & I/O Multiplexing

- **Section**: Linux System Programming
- **Topic**: Socket Programming - TCP/UDP

---

1. OSI model

osi_model.png

Layer	Name	Role	Examples
7	Application	User-facing protocols	HTTP, FTP, MQTT, CoAP
6	Presentation	Data formatting, encryption, compression	TLS, JPEG, ASCII/EBCDIC
5	Session	Session establishment/teardown, checkpointing	RPC, sockets session mgmt
4	Transport	End-to-end delivery, reliability, flow control	TCP, UDP
3	Network	Logical addressing, routing	IP, ICMP
2	Data Link	Framing, MAC addressing, error detection on a link	Ethernet, Wi-Fi (802.11), PPP
1	Physical	Bits on the wire/air	Voltages, radio, fiber optics

---

2. BSD Networking History

unix_family_tree.svg

bsd_net_history.png

- 1969 — Unix originates at AT&T Bell Labs (Ken Thompson, Dennis Ritchie).
- 1971 — First edition of Unix released internally.
- 1970s — Unix source distributed to universities, including UC Berkeley.
- 1977 — First Berkeley Software Distribution (1BSD), built on top of AT&T Unix.
- 1983 — The sockets API originated with 4.2BSD.
- 1989 — Berkeley released the first BSD networking release, containing networking code not constrained by the AT&T Unix source license, and eventually distributed via anonymous FTP.
- 1990 — 4.3BSD Reno brought OSI protocol changes to the sockets API.
- 1994 — 4.4BSD-Lite released — the last Berkeley release requiring an AT&T Unix source license.
- 1995 — 4.4BSD-Lite2 released.
- Post-1995 → today — These final releases became the base for BSD/OS, FreeBSD, NetBSD, and OpenBSD, most of which are still actively developed

---

3. Socket

3.1. What is a Socket?

A socket is one endpoint of a two way communication link between two programs running on the network.

socket.png

3.2. How Sockets Work in Computer Networks

Sockets is created using `socket` system call. The socket provides bidirectional FIFO Communication facility over the network.

- A socket connecting to the network is created at each end of the communication.
- Each socket has a specific address.
- This address is composed of an IP address and a port number.
- Socket are generally employed in client server applications.
- The server creates a socket, attaches it to a network port addresses then waits for the client to contact it.
- The client creates a socket and then attempts to connect to the server socket.

- When the connection is established, transfer of data takes place.

socket_2.png

3.3. Two Types of Sockets

Stream Sockets `SOCK_STREAM`

Datagram Sockets `SOCK_DGRAM`

3.4. Socket Address Structures

Socket address structures are passed between user processes and the kernel to specify network endpoints (IP addresses and port numbers). Each protocol suite defines its own structure.

socket_addr_struct.png

IPv4 Socket Address Structure (`struct sockaddr_in`):

Defined in `<netinet/in.h>`

```
struct in_addr {
    in_addr_t      s_addr;      /* 32-bit IPv4 address */
                                /* network byte ordered */
};
struct sockaddr_in {
    uint8_t        sin_len;     /* length of structure (16) */
    sa_family_t    sin_family;  /* AF_INET */
    in_port_t      sin_port;    /* 16-bit TCP or UDP port number */
                                /* network byte ordered */
    struct in_addr sin_addr;    /* 32-bit IPv4 address */
                                /* network byte ordered */
    char           sin_zero[8]; /* unused */
};
```

IPv6 Socket Address Structure (`struct sockaddr_in6`)

Defined in `<netinet/in.h>`

```
struct in6_addr {
    uint8_t         s6_addr[16];    /* 128-bit IPv6 address */
```

```
                                    /* network byte ordered */
};
#define SIN6_LEN        /* required for compile-time tests */

struct sockaddr_in6 {
    uint8_t         sin6_len;       /* length of this struct (28) */
    sa_family_t     sin6_family;    /* AF_INET6 */
    in_port_t       sin6_port;      /* transport layer port# */
                                    /* network byte ordered */
    uint32_t        sin6_flowinfo;  /* flow information, undefined */
    struct in6_addr sin6_addr;      /* IPv6 address */
                                    /* network byte ordered */
    uint32_t        sin6_scope_id;  /* set of interfaces for a scope */
};
```

Generic Socket Address Structure (`struct sockaddr`):

Defined in `<netinet/in.h>`

```
struct sockaddr {
    uint8_t     sa_len;
    sa_family_t sa_family;   /* address family: AF_xxx value */
    char        sa_data[14]; /* protocol-specific address */
};
```

New Generic Socket Address Structure (`struct sockaddr_storage`):

Defined in `<netinet/in.h>`

```
struct sockaddr_storage {
uint8_t     ss_len;     /* length of this struct (implementation dependent) */
sa_family_t ss_family;  /* address family: AF_xxx value */
/* implementation-dependent elements to provide:
 * a) alignment sufficient to fulfill the alignment requirements of
 *    all socket address types that the system supports.
 * b) enough storage to hold any type of socket address that the
 *    system supports.
 */
};
```

3.5. Value-Result Arguments

When a socket address structure is passed to any socket function, it is always **passed by reference** (a pointer to the structure is passed).

`bind`, `connect`, `sendto` (Value Only):

- Pass a socket address structure from the process to the kernel.
- Arguments:
  - A pointer to the socket address structure.
  - An integer size of the struct (pass by value)
    - The kernel knows how much data to copy.

sds_proc_ker.png

`accept`, `recvfrom`, `getsockname`, `getpeername` (Value-Result):

- Pass a socket address structure from the kernel to the process.
- Arguments:
  - A pointer to the socker address structure
  - A pointer to an integer size of the struct:
    - *When functions calls*: process tells kernel to not write past the end of the structure
    - *When functions returns*: kernel tells process how much data that kernel stored in the structure

sds_ker_proc.png

3.6. Byte Ordering

3.6.1. Endianness

Endianness refers to the order in which bytes are arranged in memory. Endianness comes in two primary forms:

- **Big-endian**: Stores the most significant byte first
- **Little-endian**: Store the least significant byte first

endianness.png

3.6.2. Why does it matter?

Different CPU architectures store multibyte integers in memory using different byte orderings:

- **Little-Endian**: The low-order byte is stored at the lowest memory address (used by Intel/x86 architectures).
- **Big-Endian**: The high-order byte is stored at the lowest memory address (used by Sparc, Motorola 68k, etc.)

When programming with network, we must deal with these byte ordering differences between Host Byte Order and Network Byte Order:

- **Host Byte Order**: The native byte ordering used by the local host CPU.
- **Network Byte Order**: The standard byte ordering required by Internet protocols for header fields (e.g., IP addresses and port numbers), which is strictly **Big-Endian**

3.6.3. Byte Ordering Functions

We use the following four functions to convert between Host Byte Order and Network Byte Order.

```
#include <netinet/in.h>

uint16_t htons(uint16_t host16bitvalue);
uint32_t htonl(uint32_t host32bitvalue);
/* Both return: value in network byte order */

uint16_t ntohs(uint16_t net16bitvalue);
uint32_t ntohl(uint32_t net32bitvalue);
/* Both return: value in host byte order */
```

Example:

```
#include <stdio.h>
#include <stdint.h>
#include <sys/types.h>
#include <netinet/in.h>

int main()
{
    uint32_t host_value = 1234567890;
    uint32_t network_value = htonl(host_value);

    printf("Value:\t%u\n", host_value);
    printf("Host representation:\t%x\n", host_value);
    printf("Network representation:\t%x\n", network_value);
    printf("Host prints network value: %u\n", network_value);
```

```
    return 0;
}
```

Output:

```
Value:  1234567890
Host representation:    499602d2
Network representation: d2029649
Host prints network value: 3523384905
```

3.7. inet_aton, inet_addr, inet_ntoa Functions

These functions convert an IPv4 address from a dotted-decimal string to its 32-bit network byte ordered binary value. (e.g. `"192.168.1.1"` to `0101a8c0`)

```
#include <arpa/inet.h>

int inet_aton(const char *strptr, struct in_addr *addrptr);
/* Returns: 1 if string was valid, 0 on error */

in_addr_t inet_addr(const char *strptr);
/* Returns: 32-bit binary network byte ordered IPv4 address; INADDR_NONE if error */

char *inet_ntoa(struct in_addr inaddr);
/* Returns: pointer to dotted-decimal string */
```

3.8. inet_pton, inet_ntop Functions

These two functions are new with IPv6 and work with both IPv4 and IPv6 addresses.

The letters `p` and `n` stand for presentation and numeric.

```
#include <arpa/inet.h>

int inet_pton(int family, const char *strptr, void *addrptr);
/* Returns: 1 if OK, 0 if input not a valid presentation format, -1 on error */
```

```
const char *inet_ntop(int family, const void *addrptr, char *strptr, size_t len);
/* Returns: pointer to result if OK, NULL on error */
```

- `family`: either `AF_INET` (IPv4) or `AF_INET6` (IPv6)

---

4. TCP Connection

4.1. Three-way handshake

tcp_1.png

4.2. TCP Connection Termination

tcp_2.png

4.3. TCP State

tcp_3.png

tcp_4.png

---

5. TCP Sockets

5.1. socket Function

To perform network I/O, the first thing a process must do is call the socket function

socket_function.png

```
#include <sys/socket.h>

int socket(int family, int type, int protocol);
/* Returns: non-negative descriptor if OK, −1 on error */
```

Arguments:

family	Description
AF_INET	IPv4 protocols
AF_INET6	IPv6 protocols
AF_LOCAL	Unix domain protocols
AF_ROUTE	Routing sockets
AF_KEY	Key socket

type	Description
SOCK_STREAM	stream socket
SOCK_DGRAM	datagram socket
SOCK_SEQPACKET	sequenced packet socket
SOCK_RAW	raw socket

Protocol	Description
IPPROTO_TCP	TCP transport protocol
IPPROTO_UDP	UDP transport protocol
IPPROTO_SCTP	SCTP transport protocol

Combinations of `family` and `type` for the `socket` function:

	AF_INET	AF_INET6	AF_LOCAL	AF_ROUTE	AF_KEY
SOCK_STREAM	TCP/SCTP	TCP/SCTP	Yes		
SOCK_DGRAM	UDP	UDP	Yes		
SOCK_SEQPACKET	SCTP	SCTP	Yes		
SOCK_RAW	IPv4	IPv6		Yes	Yes

5.2. connect Function

The `connect` function is used by a *TCP client* to *establish a connection* with a *TCP server*.

```
#include <sys/socket.h>

int connect(int sockfd, const struct sockaddr *servaddr, socklen_t addrlen);

/* Returns: 0 if OK, -1 on error */
```

The client does not have to call `bind` before calling `connect`: the kernel will choose both an ephemeral port and the source IP address

Error returns:

- `ETIMEOUT`: Client retransmits SYN several times at at 0s, 6s, 24s (4.4BSD). If no response is received after a total of 75 seconds, the error is returned.
- `ECONNREFUSED`: Server responds with RST - no process listening on that port
- `EHOSTUNREACH` / `ENETUNREACH`: An intermediate router returns an ICMP "destination unreachable"

5.3. bind Function

The `bind` function assigns a local protocol address to a socket.

```
#include <sys/socket.h>


int bind(int sockfd, const struct sockaddr *myaddr, socklen_t addrlen);


/* Returns: 0 if OK, −1 on error */
```

- IP address:
  - Wildcard (`INADDR_ANY`, `"0.0.0.0"`, `0`): Kernel chooses IP address
  - Local IP address: Process specifies IP address
- Port:
  - `0`: Kernel chooses port
  - nonzero: Process chooses port

`bind` can't return the kernel-assigned port (`myaddr` argument is const) => we have to use `getsockname()` to retrieve it.

Error returns:

- `EADDRINUSE`: Address already in use

5.4. listen Function

The `listen` function is called only by a TCP server, it moves the socket from the CLOSED state to the LISTEN state.

```
#include <sys/socket.h>
```

```
int listen(int sockfd, int backlog);

/* Returns: 0 if OK, -1 on error */
```

- `backlog`: the maximum number of connections the kernel should queue for this socket.

The kernel maintains two queues:

- **Incomplete connection queue** (SYN queue):
  - Contains an entry for each SYN that has arrived from a client.
  - These sockets are in the SYN_RCVD state.
- **Completed connection queue**:
  - Contains an entry for each client with whom the TCP three-way handshake has completed.
  - These sockets are in the ESTABLISHED state.

The connection creation mechanism is completely automatic by the kernel; the server process is not involved.

5.5. accept Function

`accept` is called by a TCP server to return a connection from the front of *the completed connection queue* of the kernel.

If the completed queue is empty, the process is put to sleep

```
#include <sys/socket.h>

int accept(int sockfd, struct sockaddr *cliaddr, socklen_t *addrlen);

/* Returns: 0 if OK, -1 on error */
```

5.6. close Function

The normal Unix `close` function is also used to close a socket and terminate a TCP connection.

```
#include <unistd.h>

int close(int sockfd);

/* Returns: 0 if OK, -1 on error */
```

TCP will still try to send any queued data first, then perform the normal TCP connection termination sequence.

Calling `close()` decrements the reference count of the descriptor. When the reference count is 0, the TCP connection is terminated.

5.7. Concurrent Servers

When a client request can take longer to service, we don't want it to block other clients.

A concurrent server can handle multiple clients at the same time by `fork` a child process to handle each client.

```
listenfd = Socket(...);
Bind(listenfd, ...);
Listen(listenfd, LISTENQ);

for ( ; ; ) {
    connfd = Accept(listenfd, ...);      /* blocks until a client connects */
    if ((pid = Fork()) == 0) {
        Close(listenfd);    /* child doesn't need the listening socket */S
        doit(connfd);       /* process the request */
        Close(connfd);      /* done with this client */
        exit(0);            /* child terminates */
    }
    Close(connfd);          /* parent closes connected socket */
}
```

1. Before `accept` returns, a connection request arrives from the client:
   concurrent_server_1.png
2. After `accept` return, the connection is accepted by the kernel, and a `connfd` is created:
   concurrent_server_2.png
3. After `fork`, `listenfd` and `connfd`, are duplicated between the parent and child:
   concurrent_server_3.png
4. The parent to close the connected socket `connfd` and the child to close the listening socket `listenfd`:
   concurrent_server_4.png

5.8. getsockname and getpeername Functions

Retrieve the protocol address associated with a socket - local address (`getsockname`) or the peer's address (`getpeername`)

```
#include <sys/socket.h>

int getsockname(int sockfd, struct sockaddr *localaddr, socklen_t *addrlen);

int getpeername(int sockfd, struct sockaddr *peeraddr, socklen_t *addrlen);

/* Both return: 0 if OK, -1 on error */
```

5.9. Normal termination

In a standard TCP client-server application, a termination goes in four-packet sequence:

1. Part of process termination is the closing of all open descriptors, so the client socket is closed by the kernel. This sends a `FIN` to the server.

   > At this point:
   > - Server socket: `CLOSE_WAIT` state
   > - Client socket: `FIN_WAIT_2` state

2. The server TCP responds with an `ACK` to the client's `FIN`.
3. The server then closes the descriptors, causes the final two segments of the TCP connection termination to take place: a `FIN` from the server to the client, and an `ACK` from the client.

   > At this point, client socket enters `TIME_WAIT` state

5.10. SIGPIPE Signal

When a process writes to a socket that has received an `RST`, the `SIGPIPE` signal is sent to the process.

The default action of this signal is to terminate the process, so the process must catch the signal to avoid being involuntarily terminated.

---

6. UDP Sockets

udp.png

6.1. recvfrom and sendto Functions

```
#include <sys/socket.h>

ssize_t recvfrom(int sockfd, void *buff, size_t nbytes, int flags,
                 struct sockaddr *from, socklen_t *addrlen);

ssize_t sendto(int sockfd, const void *buff, size_t nbytes, int flags,
               const struct sockaddr *to, socklen_t addrlen);

/* Both return: number of bytes read or written if OK, –1 on error */
```

- `flags`:
  - Usually `0`
  - `MSG_PEEK`
  - `MSG_DONTWAIT`
- `from`, `to`: a socket address structure containing the protocol address of where the data is to be received/sent.
- `addrlen`: size of `from`, `to`

6.2. Lost Datagrams

UDP is not reliable.

If a client datagram is dropped by a router, or if the server's reply is lost in transit, the client blocks forever in `recvfrom()` waiting for a response that will never arrive.

A typical way to prevent this is to place a timeout on the client’s call to `recvfrom`.

6.3. connect() with UDP

Calling `connect()` on a UDP socket involves no handshake. The kernel checks for errors, records the peer's IP address and port, and returns right aways.

A *connected* UDP socket compared to an *unconnected* UDP socket:

- **Sending**: Use `write`/`send` instead of `sendto`. If calling `sendto`, the address pointer must be `NULL`, and the length 0.

- **Receiving**: Use `read`/`recv`/`recvmsg` instead of `recvfrom`. The kernel delivers only datagrams from the connected peer, so the socket talks to exactly one peer.
- **Errors**: Asynchronous errors (e.g., ICMP port unreachable) are reported to the process. Unconnected UDP sockets never receive them.

Type of socket	`write` or `send`	`sendto` that does no specify a destination	`sendto` that specifies a destination
TCP socket	OK	OK	`EISCONN`
UDP socket, connected	OK	OK	`EISCONN`
UDP socket, unconnected	`EDESTADDRREQ`	`EDESTADDRREQ`	OK

When to connect: Only when the socket talks to exactly one peer. Clients usually do this.

Calling `connect` multiple times: Unlike TCP, a UDP socket can call `connect` again to:

- Change to a new peer.
- Unconnect it, by passing an address struct's family member with `AF_UNSPEC`.

Performance:

- On an unconnected UDP socket, Berkeley-derived kernels temporarily connect, send, and unconnect for every `sendto`.
- Connecting explicitly once and then calling write repeatedly avoids this overhead.

---

7. Socket Operations

7.1. getsockopt and setsockopt Functions

```
#include <sys/socket.h>

int getsockopt(int sockfd, int level, int optname,  void *optval, socklen_t *optlen);

int setsockopt(int sockfd, int level, int optname,  const void *optval, socklen_t optlen);

/* Both return: 0 if OK, −1 on error */
```

- `sockfd`: an open socket descriptor
- `level`: which code in the system interprets the option, either the general socket code or some protocol-specific code (e.g., IPv4, IPv6, TCP, or SCTP)

- `optname`: the option being queried or set.
- `optval`: A pointer to a variable. `setsockopt` reads the new value from it, and `getsockopt` stores the current value into it.
- `optlen`: The size of that variable. It is passed by value to `setsockopt` and as a value-result argument to `getsockopt`.

Two basic types of options:

- Flags (binary options):
  - `*optval` is zero: disabled
  - `*optval` is non-zero: enabled
- Values

7.2. Generic Socket Options

Level: `SOL_SOCKET`

optname	Description
SO_BROADCAST	Permit sending of broadcast datagrams
SO_DEBUG	Enable debug tracing
SO_DONTROUTE	Bypass routing table lookup
SO_ERROR	Get pending error and clear
SO_KEEPALIVE	Periodically test if connection still alive
SO_LINGER	Linger on close if data to send
SO_OOBINLINE	Leave received out-of-band data inline
SO_RCVBUF	Receive buffer size
SO_SNDBUF	Send buffer size
SO_RCVLOWAT	Receive buffer low-water mark
SO_SNDLOWAT	Send buffer low-water mark
SO_RCVTIMEO	Receive timeout
SO_SNDTIMEO	Send timeout
SO_REUSEADDR	Allow local address reuse
SO_REUSEPORT	Allow local port reuse
SO_TYPE	Get socket type
SO_USELOOPBACK	Routing socket gets copy of what it sends

7.3. IPv4 Socket Options

Level: `IPPROTO_IP`

optname	Description	Datatype
IP_HDRINCL	IP header included with data	int
IP_OPTIONS	IP header options	byte array
IP_RECVDSTADDR	Return destination IP address	int
IP_RECVIF	Return received interface index	int
IP_TOS	Type-of-service and precedence	int
IP_TTL	TTL	int

7.4. TCP Socket Options

Level: `IPPROTO_TCP`

optname	Description	Datatype
TCP_MAXSEG	TCP maximum segment size	int
TCP_NODELAY	Disable Nagle algorithm	int

7.5. fcntl Function

`fcntl` (file control) changes settings on a file descriptor. A socket is a kind of descriptor.

```
#include <fcntl.h>

int fcntl(int fd, int cmd, ... /* int arg */ );

/* Returns: depends on cmd if OK, -1 on error */
```

- fd: a file descriptor
- cmd: a command
- arg: an optional third argument (depends on cmd)

`fcntl` for sockets:

- **Get/set flags**: `F_GETFL`, `F_SETFL`
- **Nonblocking I/O** (`O_NONBLOCK`): read/write calls returns right away instead of waiting.

- **Signal-driven I/O** (`O_ASYNC`): the system sends a SIGIO signal when the socket's status changes.
- **Socket owner**: `F_SETOWN` to choose which process (or process group) receives the `SIGIO` and `SIGURG` signals. `F_GETOWN` tells the current owner.

Safe ways to set flags with `fcntl`:

```
int flags;

/* Get the current flags */
if ((flags = fcntl(fd, F_GETFL, 0)) < 0)
    err_sys("F_GETFL error");

/* Add a flag (keeps all other flags) */
flags |= O_NONBLOCK;

/* Set the new flags */
if (fcntl(fd, F_SETFL, flags) < 0)
    err_sys("F_SETFL error");
```

---

8. I/O Multiplexing

8.1. I/O Models

There are five I/O models in Unix:

- blocking I/O
- nonblocking I/O
- I/O multiplexing (`select` and `poll`)
- signal driven I/O (`SIGIO`)
- asynchronous I/O (the POSIX `aio_` functions)

io_models.png

8.2. Blocking vs. Non-blocking I/O vs. I/O Multiplexing

8.2.1. Blocking I/O

> When a requested I/O operation cannot be completed without putting the process to sleep, put the process to sleep.

Blocking operations: `read`, `recv`, `recvfrom`, `write`, `send`, `sendto`, `accept`, `connect`.

Pros: Simple, blocking doesn't cost CPU usage.

Cons: One thread can wait on only one FD at a time. Serving many connections needs thread-per-connection, which costs memory.

8.2.2. Non-blocking I/O

> When a requested I/O operation cannot be completed without putting the process to sleep, do not put the process to sleep, but return an error instead.

Use `fcntl()` with the flag `O_NONBLOCK` to make a socket non-blocking.

Pros: The thread is never stuck, so it can handle many FDs.

Cons: It forces busy-polling, burns CPU usage.

8.2.3. I/O multiplexing

> One blocking call waits on many fds at once and returns when any of them is ready.

Pros: One thread serves thousands of connections; it sleeps (no CPU) when idle and wakes only on activity.

Cons: One slow handler stalls all connections (a slow disk read, a DNS lookup via `getaddrinfo`, a big `memcpy`, JSON parsing, a mutex wait)

8.3. select Functions

The `select()` function is used to monitor multiple sockets or file descriptors (FDs) simultaneously and check whether any FD is ready for reading, writing, or has an exceptional condition.

```
#include <sys/select.h>
#include <sys/time.h>

int select(int maxfdp1, fd_set *readset, fd_set *writeset,
           fd_set *exceptset, const struct timeval *timeout);

/* Returns: positive count of ready descriptors, 0 on timeout, -1 on error */
```

- `maxfdp1`: The highest descriptor value across all sets plus 1
- `readfds`: Set of fds to watch for readability
- `writefds`: Set of fds to watch for writability
- `exceptfds`: Set of fds to watch for exceptional conditions
- `timeval`:

  ```
  struct timeval {
      long tv_sec;
      long tv_usec;
  };
  ```

  - `NULL`: Wait forever
  - `{0,0}`: Do not wait at all - polling
  - A specific value: Wait up to a fixed amount of time

```
void FD_ZERO(fd_set *fdset);          // clear all, call before use
void FD_SET(int fd, fd_set *fdset);   // add fd to the set
void FD_CLR(int fd, fd_set *fdset);   // remove fd from the set
int  FD_ISSET(int fd, fd_set *fdset); // check if fd is "ready" (after select returns)
```

`select()` can monitor only file descriptors numbers that are less than `FD_SETSIZE` (1024)

The sets are in/out parameters: the kernel overwrites them with the result, so they must be rebuilt before every call.

Finding ready fds by scanning the bitmap (from 0 to `maxfdp1`)

8.4. pselect Functions

```
#include <sys/select.h>
#include <signal.h>
#include <time.h>

int pselect(int maxfdp1, fd_set *readset, fd_set *writeset, fd_set *exceptset,
```

```
            const struct timespec *timeout, const sigset_t *sigmask);

/* Returns: count of ready descriptors, 0 on timeout, -1 on error */
```

`pselect` contains two changes from the normal `select` function:

- `timeout` uses `timespec` struct:
  ```
  struct timespec {
      time_t tv_sec; /* seconds */
      long tv_nsec; /* nanoseconds */
  };
  ```
- `sigmask`: a pointer to a signal mask, disables the delivery of certain signals

8.5. poll Functions

```
#include <poll.h>

int poll(struct pollfd *fdarray, unsigned long nfds, int timeout);

/* Returns: count of ready descriptors, 0 on timeout, -1 on error */
```

- `fdarray`: an array of information
  ```
  struct pollfd {
      int fd;         // the socket descriptor
      short events;   // bitmap of events we're interested in
      short revents;  // on return, bitmap of events that occurred
  };
  ```
  - `events`:
    - `POLLIN`: Alert me when data is ready to `recv()` on this socket.
    - `POLLOUT`: Alert me when I can `send()` data to this socket without blocking.
    - `POLLHUP`: Alert me when the remote closed the connection.
- `timeout`:
  - `INFTIM`: Wait forever
  - `0`: Return immediately, do not block
  - `> 0`: Wait specified number of milliseconds

FD limit: system (`RLIMIT_NOFILE`/`ulimit -n`)

Finding ready fds by scanning the array `fdarray`

8.6. epoll

8.6.1. Interest List vs. Ready List

Interest list: a red-black tree of registered FDs, along with the events

> A red-black tree is a self-balancing binary search tree.
> Search, insert, delete operations: O(log n) in the worst case.

Ready list: a linked list of FDs that currently have a pending event.

8.6.2. Level-triggered vs. Edge-triggered

Imagine we have a pipe buffer of 2 KB:

```
+--------------------+
|       2 KB data    |
+--------------------+
```

We call `epoll_wait(...)` and it only read 1KB

```
+---------+
|  1 KB   |
+---------+
```

What happens if we call `epoll_wait(...)` again?

Level-triggered (LT) - “Is it ready?”

"FD is readable" => It unblocks then allow we to continue reading.

LT is the default behavior.

Edge-triggered (ET) - “Did something change?”

We may not read again, since the readiness is not changed.

To use epoll as an edge-triggered:

- use the flag `EPOLLET`
- use nonblocking file descriptors
- wait for an event only after `read()` or `write()` return `EAGAIN`.

8.6.3. Create, Control, Wait an epoll

Create a new epoll instance:

```
#include <sys/epoll.h>

int epoll_create(int size); /* legacy, size is ignore but must be > 0 */
int epoll_create1(int flags);

/* Returns an epoll fd, or -1 on error, close it with close() like any fd. */
```

Control an epoll:

```
#include <sys/epoll.h>

int epoll_ctl(int epfd, int op, int fd,
              struct epoll_event *_Nullable event);
```

```
#include <sys/epoll.h>

struct epoll_event {
    uint32_t      events;  /* Epoll events */
    epoll_data_t  data;    /* User data variable */
};

union epoll_data {
    void     *ptr;
    int       fd;
    uint32_t  u32;
    uint64_t  u64;
};

typedef union epoll_data  epoll_data_t;
```

Wait an epoll

```
#include <sys/epoll.h>

int epoll_wait(int n;
                int epfd, struct epoll_event events[n], int n,
                int timeout);
```

```
int epoll_pwait(int n;
                int epfd, struct epoll_event events[n], int n,
                int timeout,
                const sigset_t *_Nullable sigmask);
int epoll_pwait2(int n;
                int epfd, struct epoll_event events[n], int n,
                const struct timespec *_Nullable timeout,
                const sigset_t *_Nullable sigmask);

                ## 9. Combining multiplexing with threads: single-thread + select/poll/epoll
vs. multi-thread + select/poll/epoll designs, and when each is appropriate

## 10. Name & Address Conversion

### 10.1. DNS

DNS (Domain Name System) is the phonebook of the Internet, which maps names to addresses.

Common DNS record types:

|Record|Meaning|Example|
|-|-|-|
|A|name → IPv4|`example.com. IN A 93.184.216.34`|
|AAAA|name → IPv6|`example.com. IN AAAA 2606:2800:220:1:248:1893:25c8:1946`|
|CNAME|alias → canonical name|`www.example.com. IN CNAME example.com.`
|PTR|IP → name (reverse lookup)|`34.216.184.93.in-addr.arpa. IN PTR example.com.`|
|MX|mail exchanger|`example.com. IN MX 10 mail.example.com.`|
|NS|authoritative name server|`example.com. IN NS ns1.example.com.`|
|SRV|service → host + port|`_sip._tcp.example.com. IN SRV 10 60 5060 sip.example.com.`|

### 10.2. gethostbyname()/gethostbyaddr()

Get an IP address for a hostname, or vice-versa

```c
#include <sys/socket.h>
#include <netdb.h>

struct hostent *gethostbyname(const char *name);
struct hostent *gethostbyaddr(const char *addr, int len, int type);
```
```

Only support IPv4.

Deprecated by POSIX.

9. Combining multiplexing with threads

10. Name & Address Conversion

10.1. DNS

DNS (Domain Name System) is the phonebook of the Internet, which maps names to addresses.

Common DNS record types:

Record	Meaning	Example
A	name → IPv4	`example.com. IN A 93.184.216.34`
AAAA	name → IPv6	`example.com. IN AAAA 2606:2800:220:1:248:1893:25c8:1946`
CNAME	alias → canonical name	`www.example.com. IN CNAME example.com.`
PTR	IP → name (reverse lookup)	`34.216.184.93.in-addr.arpa. IN PTR example.com.`
MX	mail exchanger	`example.com. IN MX 10 mail.example.com.`
NS	authoritative name server	`example.com. IN NS ns1.example.com.`
SRV	service → host + port	`_sip._tcp.example.com. IN SRV 10 60 5060 sip.example.com.`

10.2. gethostbyname()/gethostbyaddr()

Get an IP address for a hostname, or vice-versa

```
#include <sys/socket.h>
#include <netdb.h>
```

```
struct hostent *gethostbyname(const char *name);
struct hostent *gethostbyaddr(const char *addr, int len, int type);
```

Only support IPv4.

Deprecated by POSIX.

10.3. getaddrinfo

Get information about a host name and/or service and load up a `struct sockaddr` with the result

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

int getaddrinfo(const char *nodename, const char *servname,
                const struct addrinfo *hints,
                struct addrinfo **res);

void freeaddrinfo(struct addrinfo *ai);

const char *gai_strerror(int ecode);

struct addrinfo {
    int ai_flags;             // AI_PASSIVE, AI_CANONNAME, ...
    int ai_family;            // AF_xxx
    int ai_socktype;          // SOCK_xxx
    int ai_protocol;          // 0 (auto) or IPPROTO_TCP, IPPROTO_UDP

    socklen_t ai_addrlen;     // length of ai_addr

    char *ai_canonname;       // canonical name for nodename
    struct sockaddr *ai_addr; // binary address
    struct addrinfo *ai_next; // next structure in linked list
};
```

- `nodename`: hostname or IPv4/IPv6 address
- `servname`: port number ("80", "8080") or service name (“http”, “tftp”, “smtp”, “pop”)
- `hints`:

    - A pointer to an `addrinfo` that filters what you want back.
    - Optional, can be `NULL`.
    - `memset()` before use.
    - Only four fields are read from `hints`: `ai_flags`, `ai_family`, `ai_socktype`, `ai_protocol`
- `res` (output):
    - On success, `*res` points to the head of a heap-allocated linked list of addrinfo structs, one per usable address/socktype combination. Must be freed with `freeaddrinfo`.

11. Lab

Lab 1

Write a concurrent TCP echo server (fork-per-connection) plus a client

Link to lab 1

Result:

Open the server with port 8080:

```
$ ./server
Server IP: 0.0.0.0
Server Port: 8080
```

Connect 3 clients to the server with port 3001, 3002, 3003, respectively:

```
$ ./client 3001
```

```
$ ./client 3002
```

```
$ ./client 3003
```

```
$ ./server
...
[127.0.0.1:3001] Connected
[127.0.0.1:3002] Connected
[127.0.0.1:3003] Connected
```

Send messages from each clients:

```
$ ./server
...
[127.0.0.1:3001] sent 6 bytes: Hello

[127.0.0.1:3002] sent 3 bytes: Hi

[127.0.0.1:3003] sent 10 bytes: Hiiiiiiii

[127.0.0.1:3001] sent 13 bytes: How are you?
```

```
$ ./client 3001
Connecting to the server 127.0.0.1:8080
Connected to the server!
Hello
Server echo: Hello
How are you?
Server echo: How are you?
```

```
$ ./client 3002
Connecting to the server 127.0.0.1:8080
Connected to the server!
Hi
Server echo: Hi
```

```
$ ./client 3003
Connecting to the server 127.0.0.1:8080
Connected to the server!
Hiiiiiiii
Server echo: Hiiiiiiii
```

Close 3 clients:

```
$ ./server
...
[127.0.0.1:3001] Closed connection
[127.0.0.1:3002] Closed connection
[127.0.0.1:3003] Closed connection
```

Revision #8
Created 2026-09-28 02:25:36 UTC by Dung Vu
Updated 2026-10-12 01:35:49 UTC by Dung Vu